

PRODUCT REQUIREMENTS DOCUMENT

Autonomous Logistics Exception Resolution Agent

Understand the mess. Act on it. Prove it moved.

Citta RISE — Idea2Agent Edition · Problem 06

Document type: Engineering PRD (high-level)

Status: Built & verified · Version 1.0

Contents

1. Executive Summary
2. Problem, Goals & Non-Goals
3. Users & Primary Use Cases
4. Scope — Exception Types (v1)
5. System Architecture
6. Functional Requirements
7. Reliability & Safety Design
8. Data Model (v1 simulated system)
9. Tool Surface (Agent Action API)
10. Non-Functional Requirements
11. Operator Console & Observability
12. Success Metrics
13. Risks & Mitigations
14. Roadmap Beyond v1

1. Executive Summary

Logistics operations are drowning in **exceptions** — damaged packages, bad addresses, carrier outages, failed deliveries — that arrive as unstructured noise across emails, webhooks, documents and photos. Resolving each one today means a human reading the mess, checking multiple systems, coordinating with carriers and customers, and performing several actions before a shipment can move again. Fixed workflow automation cannot cover the variability; an advisory chatbot does not move the shipment.

This product is an **autonomous exception-resolution agent** that ingests a raw real-world artifact, diagnoses the exception, **dynamically** selects the right resolution path, executes real state-changing actions, and — critically — **verifies** the outcome by re-reading authoritative state and proving exactly which fields changed. It retries transient failures, escalates permanent ones to a human, and never claims success it cannot prove.

The core differentiator, stated plainly: ***most agents take an action and claim victory; this one re-reads the world and proves it moved.***

PRODUCT THESIS

Verifiable execution is the product. The agent's value is not the recommendation it makes but the state change it causes and proves — a field-level before/after diff against authoritative state, or an explicit escalation when acting would be unsafe.

2. Problem, Goals & Non-Goals

2.1 Problem statement

Each logistics exception has a different cause and circumstance. A rules engine can encode a handful of them, but the long tail — a carrier email with a buried tracking id, a webhook with inconsistent fields, a photographed damage slip — defeats fixed logic. The operational cost is human investigation time, inconsistent resolutions, and shipments stalled while a person works across systems.

2.2 Goals

- **G1 — Autonomy over advice.** The agent performs real actions that produce a verifiable operational outcome, not a summary of what should be done.
- **G2 — Dynamic resolution.** Tool selection is driven by the situation, not a predefined sequence; different exceptions take demonstrably different paths.
- **G3 — Verifiable execution.** Every resolution is confirmed by re-reading authoritative state and surfacing a measurable, field-level change.
- **G4 — Reliability under failure.** Transient failures are retried; permanent failures are recognized in code and escalated, never masked.
- **G5 — Safe autonomy.** Actions with financial or operational consequence pass through gating / human oversight before they touch the outside world.
- **G6 — Efficiency.** Model calls, latency and cost are controlled through provider failover, indexed lookups and compact context.

2.3 Non-goals (v1)

- Production carrier/TMS integrations — v1 runs against a simulated logistics system with one real external data call (geocoding).
- Multi-tenant auth, RBAC and user accounts — out of scope; the operator is a single ops role.
- A bespoke web frontend — v1 ships an operator console on the existing app layer; a production UI is a follow-on.
- Full carrier-network optimization / routing algorithms — the agent resolves exceptions, it does not plan networks.

3. Users & Primary Use Cases

User	Need	What the agent does
Ops exception handler	Clear the daily exception queue without manual system-hopping	Auto-diagnoses and resolves routine exceptions; surfaces only the ones needing judgment
Ops team lead	Consistency and an audit trail across handlers	Every action logged with before/after state and resolution status
Customer-facing team	Timely, accurate customer updates	Fires a real notification only after the fix is verified
Risk / finance owner	No unauthorized high-value actions	Gating + escalation on financially significant actions

Primary use case: an exception artifact lands (email, webhook, or image). The agent ingests it, resolves it end-to-end where safe, verifies the outcome, notifies the customer, and returns a structured resolution report — or escalates to a human with full context when autonomous action is inappropriate.

4. Scope — Exception Types (v1)

v1 addresses three exception classes, deliberately chosen to span three input modalities and three distinct resolution paths, so that dynamic tool selection is demonstrated rather than asserted.

Class	Input modality	Resolution path (happy)	Failure behavior demonstrated
Weather hold	Carrier email (unstructured prose)	Reroute to an available carrier	—
Bad address	JSON webhook (messy, inconsistent fields)	Validate → correct → reschedule → goodwill credit	Transient retry → recovers
Carrier outage	Photo of a damage/exception label (vision)	Attempt reassign → escalate to human	Permanent fail-fast → escalate

The three paths share no fixed sequence. No single hard-coded workflow can resolve all three, which is the concrete proof of **dynamic adaptability**.

5. System Architecture

The system is a layered agent: an LLM foundation with provider failover, an ingestion/extraction layer, a hand-rolled agent loop, a resilient execution layer over the logistics system, and a verify-and-diff layer that closes every loop. Every layer is inspectable — there is no black-box orchestration framework.

5.1 Layered view

Layer	Responsibility	Key components
L0 · LLM foundation	Model access with automatic failover; provider-agnostic tool-calling	chat() · Gemini → Groq failover · schema normalization
L1 · Ingestion / extraction	Turn a raw artifact into a structured exception under ambiguity	parse_exception (LLM + regex net) · extract_from_document (vision + OCR cache)
L2 · Agent loop	Plan → tool → observe → verify → answer; enforce invariants	run_agent() · policy prompt · 3 deterministic guards
L3 · Resilient execution	State changes with idempotency, retry, failure classification	with_retry · idempotency keys · transient/permanent classes · logistics DB
L4 · Verify & diff	Re-read authoritative state, prove the change, or escalate	verify_shipment · snapshot/diff engine · audit log

5.2 End-to-end flow

```

Raw artifact (email | JSON webhook | photo)
  |
  v
[ L1 parse_exception ] LLM extraction + regex safety-net
  vision path -> extract_from_document -> OCR (live, cache fallback)
  | { shipment_id, exception_type, fields, confidence }
  v
[ L2 get_shipment ]   diagnose from live, indexed state
  |
  v
[ L2/L3 DYNAMIC RESOLUTION ] policy prompt selects tools per case:
  reroute | validate_address(REAL) + update + reschedule + credit | reassign
  with idempotency + exponential-backoff retry
  transient failure -> retry -> recover
  permanent failure -> fail fast
  |
  v
[ act vs escalate ]   failure-class + value threshold decide
  |                   \__ escalate_to_human (terminal)
  v
[ L4 verify_shipment ] re-read state -> before/after diff <<< DIFFERENTIATOR
  |                   RESOLVED / ESCALATED badge
  v
[ notify_customer ]   real WhatsApp/email, gated by LIVE ACTIONS
  |
  v
Resolution report: diff + audit timeline + status

```

5.3 Design principles

- **Deterministic core, probabilistic edge.** The LLM handles extraction and tool selection; state, failure classification, idempotency and diffing are deterministic code. Reasoning lives at the edge; correctness lives in the core.
- **Verify or it didn't happen.** No resolution is reported without a re-read of authoritative state. A guard makes this non-skippable.
- **Fail loud, recover locally.** Every tool returns a structured result and never raises; the loop adapts on failure rather than terminating.
- **One real external dependency.** At least one node is ground truth the system does not control (live geocoding), with a documented fallback so availability never blocks the flow.

6. Functional Requirements

Requirements map directly to the seven required agent capabilities. Each is testable and each has an owning component.

ID	Requirement	Owning component
FR-1	Understand the exception. Extract shipment id + exception type from unstructured input across ≥ 3 modalities.	parse_exception, extract_from_document
FR-2	Gather context. Retrieve shipment, order, customer, carrier and inventory state on demand.	get_shipment, list_carriers, check_inventory
FR-3	Dynamic tool selection. Choose the resolution toolset from the situation, not a fixed order.	policy prompt + TOOLS registry
FR-4	Execute actions. Perform real state mutations: reroute, update address, reschedule, credit, reassign, escalate.	state-changing tools + logistics DB
FR-5	Validate execution. Re-read authoritative state and produce a field-level before/after diff + resolved flag.	verify_shipment, diff engine
FR-6	Handle failures. Retry transient failures with backoff; fail-fast on permanent; never terminate the workflow abruptly.	with_retry, failure classes
FR-7	Escalate. Recognize when autonomy is unsafe/insufficient and hand off to a human with full context.	escalate_to_human, thresholds
FR-8	Real notification. Notify the stakeholder via a real channel, only after verification, gated for safety.	notify_customer + LIVE ACTIONS
FR-9	Execution visibility. Surface exception, tools used, each action's result, resolution status, and human-needed flag.	trace + audit timeline UI

7. Reliability & Safety Design

7.1 Idempotency

Every state-changing tool derives a **canonical idempotency key** from its semantic arguments inside the tool (not model-supplied). A repeated call returns the cached result flagged `idempotent_replay` with no second mutation — protecting against the model re-issuing an action and against retry double-writes.

7.2 Retry & failure classification

A bounded exponential-backoff wrapper (`with_retry`, $\sim 0.2/0.4/0.8s$) wraps state changes. Failures are typed in code: **transient** failures are retried and recover; **permanent** failures fail fast and force escalation. The act-vs-escalate decision is therefore driven by a code-level failure class, not an LLM guess.

7.3 Human oversight & gating

Outbound, real-world actions (`notify_customer`) are gated behind a `LIVE ACTIONS` switch defaulting to OFF, so nothing real fires by accident. Financially or operationally significant actions escalate to a human when a value threshold or a permanent failure is reached. All logistics mutations hit the simulated system only and are always safe to run.

7.4 Loop guards (safety net)

- **Duplicate-failure guard.** A permanently-failed call cannot be re-issued; the loop short-circuits to escalation.
- **Verify-before-finish guard.** If a state change occurred, the agent cannot produce a final answer without calling verify first.
- **Notify-before-finish guard.** The real stakeholder notification cannot be silently dropped from the final step.

8. Data Model (v1 simulated system)

The logistics system is an in-memory store reset per run for determinism and reproducibility, with auxiliary indexes for O(1) lookups and an append-only audit log.

Entity	Key fields
shipments	id, order_id, status, carrier, current_hub, destination, eta, exception_code, attempts, _fail_mode
orders	id, customer_id, value_usd, items[], priority
customers	id, name, phone, email, address, preferred_channel
carriers	id, name, regions[], capacity_left, api_status, avg_delay_hrs
inventory	sku, warehouse, qty
audit_log	seq, tool, args, before, after, ok, attempts, failure_class

A monotonic seq counter replaces wall-clock time for full determinism. The audit_log is the substrate for both the verify-diff and the operator-visible timeline.

9. Tool Surface (Agent Action API)

Tools follow a strict contract: a plain function that never raises and returns a small structured dict. Schemas are constrained (enums for channels, reasons, carriers) to stay within the model's supported schema subset and to make selection legible.

Tool	Kind	Purpose
parse_exception	Ingest	Extract structured exception from raw input
extract_from_document	Ingest	Vision OCR for image inputs (cache fallback)
get_shipment	Read	Full shipment + order + customer context
list_carriers	Read	Serviceable carriers with health/capacity
check_inventory	Read	Stock lookup for reship decisions
validate_address	Read (REAL)	Live OpenStreetMap Nominatim geocoding + fallback
reroute_shipment	Write	Reroute around a hold to a new carrier
update_address	Write	Apply a validated/corrected address
reschedule_delivery	Write	Re-dispatch a failed delivery (transient-retry path)
reassign_carrier	Write	Reassign carrier (permanent-failure path)
issue_credit	Write	Apply goodwill credit to an order
escalate_to_human	Terminal	Hand off with full context when autonomy is unsafe
verify_shipment	Verify	Re-read state, compute before/after diff, resolved flag
notify_customer	Act (gated)	Real WhatsApp/email after verification

10. Non-Functional Requirements

Attribute	Target / approach
Efficiency	Provider failover (Gemini→Groq); indexed O(1) lookups; observations truncated to keep context compact; retry counted as one internal call, not loop steps
Latency	Backoff delays kept sub-second (0.2/0.4/0.8s) so recovery never stalls the operator
Reliability	Every tool non-raising; graceful fallbacks on both vision (OCR cache) and geocoding (mock); loop bounded at 12 iterations
Determinism	Per-run state reset; monotonic seq instead of wall-clock; failure injection is deterministic and reproducible
Observability	Structured per-step trace (thought, tool, args, observation, provider, ok) + append-only audit log surfaced in the console
Portability	Runs fully on the fallback provider alone; no dependency on any single model being available

11. Operator Console & Observability

The console makes the agent's work auditable in real time — the spec's visibility requirements are first-class UI, not logs.

- Raw artifact panel — the exact email / webhook / photo the agent received.
- Live agent trace — each step as a card: thought, tool, arguments, observation, serving provider, and ok/failure chip.
- **Before/after diff table** — the verify step rendered as red (old) → green (new) field changes, with a **RESOLVED** or **ESCALATED** badge.
- Resilience audit timeline — every mutation with attempt count and failure class (transient/permanent).
- **LIVE ACTIONS** toggle — operator control over whether real notifications fire.

12. Success Metrics

Metric	Definition	v1 target
Autonomous resolution rate	Exceptions resolved without human escalation	≥ 2 of 3 classes fully autonomous
Verification coverage	Resolutions confirmed by a state re-read	100% (guard-enforced)
False-success rate	Reported resolved but state unchanged	0% (verify-diff makes it impossible)
Recovery rate	Transient failures recovered without human	100% of transient cases
Unsafe-action rate	High-consequence actions taken without gate/escalation	0%
Provider resilience	Scenarios completing on fallback provider alone	100%

13. Risks & Mitigations

Risk	Impact	Mitigation
LLM extraction fails on messy input	Wrong shipment/exception identified	Regex safety-net recovers id + type when model output is unusable
Vision provider quota / outage	Image scenario can't OCR	Live-first with on-disk OCR cache fallback; pre-seeded so it never blocks
External geocoder latency/down	Address validation stalls	Short timeout + mock fallback; never on the critical mutation path
Model narrates an action instead of calling it	Real action silently skipped	Content-free history echo + notify/verify guards force the real call
Loop wandering exceeds budget	Recovery scenario truncated	MAX_ITERS=12; duplicate-failure guard kills spirals; retry is one internal call
Accidental real send on stage/prod	Unwanted customer contact	LIVE ACTIONS defaults OFF; only notify_customer touches the outside world

14. Roadmap Beyond v1

- **Real integrations.** Swap the simulated system for live carrier/TMS/WMS APIs behind the same tool contract — the agent loop is unchanged.
- **Multi-agent decomposition.** Split ingestion, resolution and verification into specialized agents for throughput and clearer ownership.
- **Learning from escalations.** Feed human resolutions of escalated cases back as new resolution patterns.
- **Approval workflows.** Formal human-in-the-loop approval queues for high-value actions, with SLAs and audit.
- **Expanded exception coverage.** Customs holds, missing documentation, damaged-goods replacement with inventory allocation.

Appendix — Reference scenarios (verified): A weather-hold email → reroute → verify → notify (7 steps) · B messy webhook → real geocode → update → reschedule[retry ×1] → credit → verify → notify (9 steps) · C damage photo → vision → reassign[permanent] → escalate → verify → notify (9 steps).